WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Created by the Aerospace Cyber Defence Centre of Excellence (CEC)

# Testing guide
# WAC (Windows Artefacts Collector)

## Windows artefact collection software for digital forensic investigation



SYSTEM ARTEFACTS

USER FILES

DIGITAL EVIDENCE

A TOOL IN SUPPORT OF DIGITAL INVESTIGATION

# WAC — Windows Artefacts Collector

Testing guide

| | |
|---|---|
| Software version | 2.0.0 |
| Document edition | September 2026 |
| Publisher | Aerospace Cyber Defence Centre of Excellence (CEC) |
| | French Air and Space Force Academy, Air Base 701, Salon-de-Provence |

This document explains how to run each of the WAC tests by hand, what each one proves and does not prove, and how to read a failure. It is meant for whoever changes the code or wants to check a version before using it. The durations and volumes quoted were measured on the test virtual machine and on a real collection; they are orders of magnitude. The results of these tests, and what they guarantee, are in the *Validation report*.

# Contents


**1 Principles** **3**

1.1 What a WAC test must prove . . . 3

1.2 Three rules . . . 3

**2 Setting up** **4**

2.1 The Linux workstation . . . 4

2.2 Building WAC and the harnesses . . . 4

2.3 The test virtual machine . . . 5

**3 The test cycle in the virtual machine** **6**

3.1 Running a cycle . . . 6

3.2 Checking the outputs . . . 7

3.3 Comparing two runs . . . 9

**4 The format harnesses** **10**

4.1 Fingerprints: `sha_test` . . . 10

4.2 JSON reading: `json_test` . . . 10

4.3 Decompression: `lznt1_test` and `xpress_test` . . . 10

4.4 Artefacts by version of Windows: `artefact_catalogue_test` . . . 11

4.5 Browser sessions: `browser_sessions_test` . . . 12

4.6 LZX decompression: `lzx_test` . . . 12

4.7 Event messages: `wevt_test` and `event_messages_test` . . . 12

4.8 Event logs: `evtx_test` . . . 13

4.9 Hive reader: `offline_registry_test` . . . 13

4.10 Conversions: `system_conversions_test` . . . 13

4.11 Exhibit store: `consigne_test` . . . 14

4.12 Replay of the hive logs: `hive_recover_test` . . . 14

4.13 Hashed key of the SAM: `sam_secrets_test` . . . 15

4.14 Shadow copies read raw: `vss_test` . . . 15

4.15 Verification scripts: `verify_exhibits_test.sh` . . . 16

4.16 Raw reading: `raw_hive_test` . . . 16

4.17 NTFS records: `ntfs_fixup_test` . . . 16
4.18 NTFS data runs: `ntfs_runs_test` . . . 16
4.19 Deleted files: `deleted_files_test` . . . 17
4.20 Carving: `carving_test` . . . 17
4.21 `$MFT` and change journal: `mft_test`, `usn_journal_test` . . . 18
4.22 Raw reading on a real volume: `raw_volume_test.sh` . . . 18
4.23 Outlook's mail: `pst_test` . . . 19
4.24 Internet Explorer's indexes: `ie_index_dat_test` . . . 19
4.25 Windows Timeline: `activities_cache_test` . . . 19
4.26 Small file artefacts: `file_artefacts_test` . . . 20
4.27 Signatures: `authenticode_test` . . . 21
4.28 Configuration: `yaml_light_test`, `config_test` . . . 21
4.29 Third-party chains: `third_party_chain_test` . . . 21
4.30 Vulnerable drivers: `version_info_test`, `vulnerable_drivers_test` . . . 22
4.31 Trust lists: `inflate_test`, `cab_test`, `zip_test`, `csv_reader_test`, `trust_list_test` 22

**5 The robustness harnesses 24**
5.1 The principle . . . 24
5.2 Shortcuts: `lnk_test` . . . 24
5.3 Jump lists and Prefetch: `parsers_test` . . . 24
5.4 Reference . . . 25

**6 Cross-checking on a real collection 26**

**7 Reading a failure 27**
7.1 Exit codes . . . 27
7.2 Locating a read outside the buffer . . . 27
7.3 An endless loop . . . 27

CHAPTER 1

# Principles

## 1.1 What a WAC test must prove

WAC produces JSON files. That they are valid proves almost nothing: a field read at the wrong offset, a date interpreted in the wrong time zone, a name decoded with the wrong code page all give perfectly valid JSON — and wrong JSON. The costliest defects found in WAC were all of that kind: they showed only when a value was confronted with another source.

The tests therefore fall into three families, which do not answer the same question:

**The virtual machine cycle** (chapter 3): does WAC run to the end on a real Windows, and are its outputs consistent with each other? The *cross-checks* confront a value computed by WAC with an independent value from the same collection.

**The format harnesses** (chapter 4): does a decoder give exactly the result of the reference — Microsoft's own tool, a published standard, another implementation?

**The robustness harnesses** (chapter 5): does a parser read outside its buffer, or loop for ever, on a truncated or forged file?

## 1.2 Three rules

1. **A fix comes with the test that would have caught the defect.** And that test is checked both ways: it passes on the fixed output, and it fails when the defect is replayed.
2. **After any change, compare with the previous run**, not just the green marks: an artefact that went from 1,522 entries down to 573 lights no warning at all (section 3.3).
3. **Real data does not leave the test workstation.** A real collection holds personal data: it is processed locally, its temporary copies are deleted after use, and none of it enters the repository (`vmtest/results/` is ignored by git).

CHAPTER 2

# Setting up

## 2.1 The Linux workstation

Everything is built on Linux, cross-compiled with MinGW-w64; the harnesses run natively, under `wine`, or in the virtual machine.

```
sudo apt install g++-mingw-w64-x86-64 wine64 python3 python3-pip
pip install olefile          # independent OLE reader, for the cross-checks
pip install dissect.esedb    # independent ESE reader: WebCache (check skipped otherwise)
# or, leaving the system's Python alone: vmtest/tools/ese-venv
python3 -m venv vmtest/tools/ese-venv && vmtest/tools/ese-venv/bin/pip install dissect.
    esedb==3.18
```

## 2.2 Building WAC and the harnesses

```
./build-windows.sh --test
```

| **Output, in `build-windows/`** | **Role** |
|---|---|
| `WAC.exe` | the collection tool |
| `raw_hive_test.exe` | raw NTFS reader, used by the VM cycle |
| `tests/lnk_test.exe` | robustness of the shortcut parser |
| `tests/parsers_test.exe` | robustness of the jump list and Prefetch parsers |
| `tests/evtx_test.exe`, `consigne_test.exe` | event log decoder, exhibit store (chapter 4) |
| `tests/offline_registry_test.exe` | hive reader confronted with Microsoft's |
| `tests/system_conversions_test.exe` | WAC's conversions confronted with Windows' |
| `tests/ntfs_fixup_test.exe` | check of NTFS records (torn records) |
| `tests/ntfs_runs_test.exe` | NTFS data runs (a forged list refused) |
| `tests/deleted_files_test.exe` | recovery of deleted files on a volume in memory |
| `tests/carving_test.exe` | exact size of a carved PE, ZIP, PDF |

| Output, in `build-windows/` | Role |
|---|---|
| `tests/mft_test.exe`, `usn_journal_test.exe` | `$MFT` and change journal against ntfs-3g and the compiler's structures |

> **Warning**
> The harnesses are deliberately in `build-windows/tests/`, away from `WAC.exe`: they are test tools, with no business on a collection medium. Never copy the `tests/` folder onto one.

The build must end without a single warning: `-Wall -Wextra` are on, and warnings have already pointed at real defects.

The other harnesses (chapter 4) are each built with a command given in their header and repeated in this guide; several compile natively on Linux.

## 2.3 The test virtual machine

The VM is a Windows 11 driven through the *qemu-guest-agent*: no interaction, no network, commands run as `NT AUTHORITY\SYSTEM`. Its creation and prerequisites are described in `vmtest/README.md`; in short:

```
cd vmtest
ISO_WIN=~/Downloads/Win11_25H2_English_x64.iso ./create-vm.sh   # once
./install-browsers.sh                          # Chrome and Firefox, for web_client (once)
python3 lib/qga.py ping                        # does the agent answer?
python3 lib/qga.py run --shell "dir C:\\"      # run a command
python3 lib/qga.py write <host> <guest>        # send a file
python3 lib/qga.py read  <guest> <host>        # fetch a file
```

> **Keep in mind**
> A command started with `qga.py run` is cut after 30 minutes. For a longer test (for instance the Prefetch files of a real machine, section 5.3), start it as a detached task in the VM and follow its log.

CHAPTER 3

# The test cycle in the virtual machine

## 3.1 Running a cycle

```
cd vmtest
./run-wac-test.sh --build     # rebuild, then full cycle
./run-wac-test.sh             # full cycle with the binaries already built
./run-wac-test.sh --raw-only  # only the validation of the raw reading
./run-wac-test.sh --no-split  # without step 7 (collection then conversion)
./run-wac-test.sh --binary-all  # step 7 with --binary-all
./run-wac-test.sh --only=mft,usn_journal  # only these artefacts
```

`--only` takes keys of the `artefacts:` section of `wac.yml`: the harness places next to `WAC.exe` a `wac.yml` that switches every other one off, for a cycle a few minutes long when a single artefact changed. The checks of an artefact switched off are skipped (its JSON says `Not Requested`); the artefacts read in the hives also need `registry`, and step 8 (`wac.yml`) is skipped. A full cycle remains the rule before a commit that touches the core.

A second fixed disk of 2 GB (`create-vm.sh`) is formatted NTFS on the first cycle, label `WACDATA`: it holds the `$MFT` and journal of a volume other than the system's, with their witnesses.

The cycle chains: upload of `WAC.exe` and `raw_hive_test.exe` into the VM, validation of the raw extraction of a hive (`reg load` must refuse it until it is repaired, then accept it), a collection by WAC then the conversion of that collection — WAC never does both in the same run —, then fetch, into the folder `vmtest/results/<timestamp>/`, of:

- `run.log` — the console output of the collection, which ends with `END, Time elapsed` when WAC went to the end;
- `raw.log` — the trace of the NTFS reader;
- the JSON files of the artefacts;
- `exhibits/` — the exhibit manifest, its seal and the list of the files really present in the exhibit store.

Step 7 then separates collection and conversion (chapter "Collect here, convert elsewhere" of the user documentation): `--collect --events --binary`, which must convert nothing; a conversion of an exhibit store with one retouched exhibit, which must be refused before any read, with its reason; a conversion, whose JSON files pass the checks below; a second conversion, whose JSON files must be identical to the first's, byte for byte. Results in `split/` and `split-again/`.

Around this collection, the cycle checks that the reading is **harmless**: `vmtest/guest/harmlessness.ps1` records, before and after, the three dates and the USN number of two witness files that `--binary` collects (a macro document, a script whose dates are forged to

2001). They must be identical. A read of the script through Windows (`Get-Content`) must then change its access date: otherwise the check would prove nothing about the dates. The records are in `harmlessness.txt`; the approach is in the *Harmlessness file*. Before this collection, the cycle waits 90 s after its `taskkill`: Windows writes a registry change to the disk a minute or so after making it, and the BAM check looks for it in the hive read from the disk.

### Windows XP

XP has neither guest agent nor PowerShell: `run-wac-test-xp.sh` drives it by the keyboard and its serial port.

```
./run-wac-test-xp.sh winxp-test      # XP SP1; winxpsp3-test for SP3
```

A CD image carries `WAC32.exe`, the `wac.yml` of `bin/` and `run-xp.cmd`, which the script types into the Run box: it collects, converts, says each step and its exit code on `COM1`, deletes `WAC32.exe` and shuts the machine down. The JSON, the manifest and the logs are then read on the disk of the VM, off, read-only (`lib/read-offline.py`, dissect), and checked by `check-json.py`. Without PowerShell no witness is placed: the checks bear on the outputs' consistency and on what the version must have or not (IPv4-only network tables, Prefetch 17 without `$MFT` reference, `Thumbs.db` without checksum). `run-xp.cmd` spans several lines on purpose: it checks that WAC gives the console its code page back, without which the `cmd.exe` of before Windows 8 stops reading a batch file after it.

## 3.2 Checking the outputs

```
python3 lib/check-json.py results/<timestamp>
```

Each line starts with a green tick (check passed), a red cross (defect) or a "skip" sign (check not applicable: the artefact is absent or empty). The exit code is 0 if no check failed. Beyond JSON validity and the shape of paths, the script confronts independent sources:

| Cross-check | What it has already caught |
|---|---|
| Prefetch hash / suffix of its file name | 65 wrong hashes out of 270 (hexadecimal without padding) |
| Prefetch executable name / start of the file name | a name read 8 bytes too early ("\x11" everywhere) |
| Prefetch `\VOLUME{guid}` paths resolved | 20,066 paths out of 20,066 unresolved |
| `X` / `XUtc` pairs, lists included | double time-zone shift; Prefetch `RunsUtc` 2 h off |
| offset of every local date / tz database | the collection day's offset applied to every date; daylight saving dates misread |

| Cross-check | What it has already caught |
|---|---|
| no session earlier than the boot | date shifts, a boot time 3.5 s late |
| ShimCache date / NTFS date in the manifest | a UTC date treated as local (-2 h) |
| DestList entries / jump list shortcuts | the first 15 entries of every list lost |
| machine name of the events / `OperatingSystem.json` | check of the BinXML decoding |
| record identifiers unique per log | a stale chunk read again |
| mounts / `reg query` and `Get-Partition` | device paths published as “\” |
| precision of FAT and text dates | seven digits of fraction on dates to the second |
| BAM, UserAssist, USBSTOR, Amcache / harness, Prefetch, Windows, PE headers | UTC dates read as local (2 h) |
| SID names / `Get-LocalUser`, `Get-LocalGroup` | names asked of the system, translated (“Système”) |
| GUIDs registered by the harness (`register-test-guids.ps1`) | “Unmapped GUID” published as a name |
| seal / real fingerprint of the manifest | a retouched manifest |
| files in the exhibit store / manifest content | 209 exhibits added after sealing |
| SHA-256 of files read raw / `Get-FileHash` | safeguard of the reading by batches of clusters |
| Prefetch dates / dates given by Windows | 0 out of 335 matching: dates of the working copy |
| executables of System32 and SysWOW64 / manifest of a `--collect --binary` | 6,446 missing out of 8,540 (`$INDEX_ROOT` in an extension record, hard links) |
| catalogs cited in a verdict / exhibits of the store | 170 out of 170 missing (label “catalog” looked for as “catalogue”) |
| WAC’s Microsoft verdicts / `Get-AuthenticodeSignature` | safeguard: a wrong verdict would leave a binary out of the store |
| copy of a Store package (untouched, modified, added) / verdicts | verification through the signed block map |
| two conversions of one collection | dates read on the working copy, rebuilt at each conversion |
| forged dates of a script / `$STANDARD_INFORMATION` and `$FILE_NAME` | safeguard: the forgery must stay visible |
| retrieval keys / PE headers read by PowerShell | safeguard of the retrievable content |

| Cross-check | What it has already caught |
|---|---|
| `--binary-all` / copy and verdict of every executable | package check too strict under `--binary-all` |
| `verify-exhibits.ps1` / separate collection | fingerprint-only exhibits reported altered |

The full list, with the history of each check, is in `vmtest/README.md`.

## 3.3 Comparing two runs

A green mark does not tell whether a piece of data disappeared. After any change, compare the new collection with the previous one: number of entries of each artefact, and values lost or gained. Two precautions:

- a collection changes by itself from one run to the next: timestamps, new events, updated Prefetch files, the fingerprint of `$MFT`. Compare *paired* items — for instance a Prefetch by its path hash and its last run — leaving aside the dates of the extracted copy;
- a value that looks “lost” may just be an empty field no longer emitted, or a renamed key: normalise before concluding.

CHAPTER 4

# The format harnesses

Each one confronts a WAC decoder with a reference. All are excluded from the WAC build by their `_test.cpp` suffix.

## 4.1 Fingerprints: `sha_test`

Confronts SHA-1 and SHA-256 with the published vectors of FIPS 180-4, and with lengths 55 to 128 bytes, which exercise the padding — the one place where an otherwise correct implementation goes wrong.

```
cd sources && g++ -std=c++17 -I. sha.cpp sha_test.cpp -o /tmp/sha_test && /tmp/sha_test
```

## 4.2 JSON reading: `json_test`

`--convert` reads back what `--collect` wrote: the exhibit manifest and the snapshot of the live state. The harness checks that everything WAC's writer produces reads back identical, that the files of a real collection read back and rewrite to the byte, that malformed texts are refused, and that 200,000 random mutations never make the reader read out of bounds (AddressSanitizer, UBSan).

```
cd sources && g++ -std=c++17 -g -fsanitize=address,undefined -I. json_test.cpp -o /tmp/
    json_test
/tmp/json_test ../vmtest/results/<timestamp>/*.json
```

## 4.3 Decompression: `lznt1_test` and `xpress_test`

Check the NTFS (LZNT1) and WOF (XPRESS Huffman) decompressors against *Microsoft's own compressor*: a known file is compressed in the VM by `RtlCompressBuffer`, unit by unit, and the harness decompresses each unit and compares it byte for byte with the original. A wrong decompressor usually decodes without any error: only the exhaustive comparison reveals it.

```
cd WAC
g++ -std=c++17 -I. lznt1.cpp lznt1_test.cpp -o /tmp/lznt1_test
g++ -std=c++17 -I. xpress.cpp xpress_test.cpp -o /tmp/xpress_test
/tmp/lznt1_test  <original> <compressed.bin> <index.idx>
/tmp/xpress_test <original> <compressed.bin> <index.idx>
```

Each unit has one line in `index.idx`: `<n> <original size> <compressed size>`; a null compressed size marks a unit Windows gave up compressing.

## 4.4 Artefacts by version of Windows: `artefact_catalogue_test`

Checks the table that says, for each version of Windows, which artefacts exist, where and in which format (`artefact_catalogue.cpp`). That table decides what a collection does *not* take: a wrong line would lose an exhibit without a word. The test checks that the table and `wac.yml` hold exactly the same keys, that each artefact's forms are in order, that known cases answer right (no SRUM before Windows 8, Prefetch 23 on Vista and 30 on Windows 10), that on Windows 7 the guard holds back only the five artefacts that version lacks, and that a Windows newer than the table is treated as the last one known, and flagged. An unknown key or version never holds a collection back. It also checks that a source is said "always there" only where it is: not on a server, not for a key that exists only once a device was plugged, not for a metadata file of the volume or a state of the running system. Lastly, that a piece is decoded only if the table holds its signature with a decoder: a Prefetch of version 23 is named and not decoded, a Prefetch of version 32, which nobody has seen yet, has neither form nor decoder. Seen failing: SRUM declared present since XP; `USBSTOR` declared always there.

```
cd sources && g++ -std=c++17 -g -fsanitize=address,undefined -I. artefact_catalogue.cpp
    artefact_catalogue_test.cpp -o /tmp/act
/tmp/act wac.yml
/tmp/act wac.yml --dump    # the table, for doc/artefact-table.py
```

In the cycle, the version WAC reads in the copied `SOFTWARE` hive is confronted with the one Windows gives of itself (build number, workstation or server).

The last step of the cycle moves the `hosts` file aside for the time of a collection of that artefact alone, then puts it back. The conversion must give `hosts.json` the status `ExpectedButMissing`, with the build Windows reports and the place where the file should have been, and record the finding. The independent reference is the manifest of the exhibit store: it holds the other files of the folder — the folder was read — and no `hosts`. No other artefact may carry the status; in the rest of the cycle, where the test machine lacks nothing, it is a fault. Seen failing on an ordinary collection.

A format without decoder is presented to the decoders themselves. `parsers_test` rewrites the version of a real Prefetch to the next one: it must be declared unsupported, the name and the hash read, nothing else decoded. `file_artefacts_test` does the same for a `$I` index of version 3, a thumbnail cache of Vista and a jump list DestList of version 5 or 7 (version 6 of Windows 11, measured with `olefile`, is read). In the cycle, a Prefetch file written by hand (version 32, never seen) is placed in `C:\Windows\Prefetch`: `prefetchs.json` must declare it without naming a format, and declare no real Prefetch — every version from XP's on is decoded. Every real Prefetch of the machine is then read by libscca, written independently of WAC: version, hash, run count, run times to the 100 ns, volume serials and number of files must be its.

## 4.5 Browser sessions: `browser_sessions_test`

Checks the decoding of Chromium's session files (SNSS: `Session_*`, `Tabs_*`) and of Firefox's mozLz4 files (`.jsonlz4`). The LZ4 blocks are made by Python's `lz4` module, an independent implementation: each must decompress byte for byte. The SNSS files are written by the test field by field, as Chromium lays them out (Pickle, fields aligned to 32 bits): a navigation replaced by a later one, an older navigation without the fields after its title, a closed tab and its instant, a truncated command, a lying length. Then every input, damaged at random (fixed seed), must never make the decoders read out of bounds (AddressSanitizer, UBSan). Seen failing: without the fields' alignment, and with an LZ4 copy that does not overlap.

```
cd sources && g++ -std=c++17 -g -fsanitize=address,undefined -I. browser_sessions.cpp
    browser_sessions_test.cpp -o /tmp/bst
python3 -c "import lz4.block,sys; open(sys.argv[2],'wb').write(lz4.block.compress(open(sys
    .argv[1],'rb').read(), store_size=False))" a.bin a.lz4
/tmp/bst a.bin a.lz4 [<original> <block> ...]
```

In the cycle, Chromium's sessions are confronted with `History`: each page the history knows carries the instant of one of its visits, to the microsecond; Firefox's, decompressed by Python's `lz4` module, must give the same pages, states and closing instants.

## 4.6 LZX decompression: `lzx_test`

WOF's fourth algorithm, LZX on 32 KiB chunks, is only produced on request. The test data come from Windows itself: in the VM, `compact /c /exe:lzx` compresses a file, and `raw_hive_test --wof` writes its compressed stream as it lies on the disk. The test decompresses every chunk and compares it with the original, then submits every chunk, truncated and damaged at random (fixed seed), to the decoder built with AddressSanitizer and UBSan: no read or write out of bounds, and never a "success" of the wrong size.

```
cd WAC
g++ -std=c++17 -g -fsanitize=address,undefined -I. lzx.cpp lzx_test.cpp -o /tmp/lzx_test
/tmp/lzx_test <original> <wof-stream.bin>
```

Reference: `ntoskrnl.exe`, `shell32.dll` and a text file, 646 chunks out of 646 identical, 235,707 checks, no failure. The VM cycle also checks, on every run, the SHA-256 of these three files compressed with LZX and read raw against `Get-FileHash`.

## 4.7 Event messages: `wevt_test` and `event_messages_test`

`wevt_test` checks the chain that turns an event into a sentence, on the files of a real provider: its DLL and its localised satellite.

```
cd sources && g++ -std=c++17 -I. pe_resource.cpp wevt.cpp wevt_test.cpp -o /tmp/wevt_test
/tmp/wevt_test gpsvc.dll fr-FR/gpsvc.dll.mui "{aea1b4fa-97d1-45f2-a64c-4d69fffd92c9}" 1002
     4001
```

`event_messages_test` does the same starting from the SOFTWARE hive; it runs on the examined machine or in the VM, the only places where the hive and the providers' binaries are readable:

```
event_messages_test <SOFTWARE hive> <guid> [id[:version] ...]
```

## 4.8 Event logs: `evtx_test`

Reads an `.evtx` outside Windows, under `wine`; built by `build-windows.sh --test`, in `build-windows/tests/`. It serves to try the decoder on logs the VM does not produce: old, forwarded, closed abruptly, damaged — and to compare record by record with an independent implementation (`python-evtx`).

```
wine evtx_test.exe "Z:/path/Security.evtx"            # summary per log
wine evtx_test.exe "Z:/path/Security.evtx" 3          # + XML of 3 records
wine evtx_test.exe "Z:/path/Security.evtx" --dump     # one record per line
wine evtx_test.exe --collect "Z:/tmp/tree" "Z:/tmp/out"          # complete chain
wine evtx_test.exe --collect-memory "Z:/tmp/tree" "Z:/tmp/out2"  # same chain, in memory
cmp /tmp/out/events.json /tmp/out2/events.json                   # must be silent
```

## 4.9 Hive reader: `offline_registry_test`

WAC reads registry hives with its own reader (`offline_registry.cpp`), linked into the executable: it no longer depends on Microsoft's `offreg.dll`, which Windows ships and which it therefore borrowed from the examined machine. This harness confronts the two: it loads Microsoft's DLL from the path given, walks each hive with both readers and compares every key (name, date, class, counts, maxima, security descriptor size) and every value (name, type, bytes), as well as the contract's edge cases: buffer too small, size probe, default value, missing key or value. No difference is allowed.

```
offline_registry_test.exe C:\Windows\System32\offreg.dll SYSTEM SOFTWARE SAM ntuser.dat
    ...
```

To be run on Windows (the VM), on replayed hives: those of a collection's `working/` folder. The one intended difference: a hive whose logs were not replayed is refused by WAC, where recent versions of Microsoft's DLL read it as is — stale keys, without an error; the harness reports it as expected. Reference: 18 hives, ten of them from a real machine, i.e. 1,273,652 keys and 2,456,139 values, identical.

## 4.10 Conversions: `system_conversions_test`

WAC imports only what querying the running system requires (`ADVAPI32`, `KERNEL32`, `msvcrt`, `Secur32`, `WTSAPI32`). The conversions it used to ask of other libraries are done by its own code; this harness confronts them with the Windows functions they replaced: GUIDs

(`StringFromGUID2`), SIDs both ways (`ConvertSidToStringSidW`, `ConvertStringSidToSidW`), OLE dates (`VariantTimeToSystemTime`) and property names (`PSGetNameFromPropertyKey`), FAT dates (`DosDateTimeToFileTime`, on every date), and daylight saving time: WAC's offset in both directions (`SystemTimeToTzSpecificLocalTimeEx`, `TzSpecificLocalTimeToSystemTimeEx`) on *every* time zone of Windows, with its rules year by year, at the transitions (hour skipped, hour repeated, change of rule on 1 January) from 1980 to 2035; the reading of `SYSTEM\...\TimeZoneInformation`, whose daylight saving dates are `TIME_FIELDS`, not `SYSTEMTIMES`; rules forged at random; the reading back of the manifest's ISO 8601 dates (`iso8601UtcToFiletime`, exact to the 100 ns on 200,000 instants, malformed dates refused). It also checks the suspect's offset in both directions: a local date converted to UTC must be so with the time zone of the examined machine, not that of the machine running WAC. An impossible date (month 13, hour 25) gives a null date, hence not emitted. The table of property names is generated from the schema of a reference Windows by `vmtest/guest/property-schema.cpp`.

```
system_conversions_test.exe
```

To be run on Windows (the VM). Reference: 9,861,570 comparisons, no difference. Two intended differences: WAC refuses a SID text Windows would truncate (a sub-authority of $2^{32}$ or more, a sixteenth sub-authority), and a NaN date, for which Windows returns a meaningless time.

## 4.11 Exhibit store: `consigne_test`

Replays the production chain — fingerprints, manifest, verified copy, hive replay — on hives given as arguments, and checks what no build reveals: the exhibit store stays identical byte for byte while the working copy is changed, and the seal carries the manifest's real fingerprint. Last, it checks that an exhibit path read from a manifest stays inside the exhibit store: a `..` component is refused, a WinSxS name holding `..` (`i..sermode`) is accepted.

```
cd build-windows/tests
wine consigne_test.exe "Z:/tmp/out" "Z:/path/SYSTEM" "Z:/path/ntuser.dat"
```

## 4.12 Replay of the hive logs: `hive_recover_test`

A replay that drops an entry, or applies a stale one, gives a hive that still opens and still reads: only the keys of the lost pages are missing, without any error. The test forges a hive and its logs where each page holds one letter and each entry writes another: the letter left in a page says which entry was applied there last. It checks that the entry numbered as the hive's sequence is applied — it used to be skipped, one entry lost at every replay —, that the earlier entries and the leftovers of a previous generation are not, that an entry growing the hive extends the file and the header, that the header ends clean with its checksum, that the undo log gives the original hive back, and that an altered entry ends the chain.

```
cd build-windows/tests
wine hive_recover_test.exe "Z:/tmp/work"
```

Reference: 11 checks, no failure; four fail if the comparison of the sequence numbers is put back to its former form. On real hives, the VM cycle confronts WAC's replay with Windows' own recovery of the same files (keys, values and a digest of them all).

## 4.13 Hashed key of the SAM: `sam_secrets_test`

A wrong constant in the SAM's RC4 scheme (Windows 8.1 and earlier, and the accounts nothing has rewritten since) gives a wrong hashed boot key, then wrong NT hashes for every account, without any error. The test builds an F value from a synthetic boot key and hashed boot key, encrypted and given its checksum by an independent computation; it checks that the hashed boot key comes back, and that a wrong boot key, an altered checksum, a truncated value or an unknown revision are refused: no key rather than a wrong one.

```
cd build-windows/tests
wine sam_secrets_test.exe
```

Reference: 5 checks, no failure; the first fails with the former constant (thirteen "Q" instead of twelve). On a real SAM, the VM cycle confronts a witness account's hash with MD4 of its password, on Windows 11 (AES) and on Windows 8.1 (RC4).

## 4.14 Shadow copies read raw: `vss_test`

A wrong rule in the application of the shadow copy stores gives a copy whose files are valid and wrong: a block of the current volume where the copy kept another. Without argument, the test builds a volume of 64 blocks and two copies whose answers are set by hand, block by block: a block the older store keeps, a forwarder to the newer store, a forwarder taken over by a later descriptor, an overlay of one sector that serves its own copy only, an ignored descriptor, an unused block read as zeros for the newer copy alone, a catalog given out of order, a block Windows copies after the lists were read (seen after a refresh, the older store kept), a catalog chain that loops (refused).

```
cd build-windows/tests
wine vss_test.exe
wine vss_test.exe Z:\path\volume.raw   # image of a volume: list and SHA-256 of each copy
```

Reference: 30 checks, no failure; seen failing on four defects put back (overlay applied to every copy, unused blocks not zeroed, forwarder chain not merged, older stores lost by the refresh). On a real 2 GiB volume holding two copies, the list and the SHA-256 of each copy's rebuilt volume are libvshadow's; the witness files Windows reads through the copy's device are the same. The VM cycle confronts the manifest's fingerprints with those Windows computes through the copy the harness made.

## 4.15 Verification scripts: `verify_exhibits_test.sh`

Runs `verify-exhibits.sh` and `verifier-consigne.sh`, the Linux scripts delivered with `WAC.exe`, on forged collections whose verdict is known: an intact collection (a path with a space, a fingerprint in lower case, a fingerprint-only binary) is accepted; an altered, missing or unfingerprinted exhibit, a modified manifest and an unreadable one are refused. A manifest that names a file of the investigation machine — by `..`, by an absolute path or by a symbolic link — is refused without the file being read. Both scripts run every case, with the same exit code and counters expected: one cannot drift from the other unnoticed. Linux only, no data needed.

```
bash sources/verify_exhibits_test.sh
```

Reference: 23 checks, no failure. The `.ps1` scripts are run by the VM cycle on the separate collection.

## 4.16 Raw reading: `raw_hive_test`

Extracts a file by reading the NTFS volume directly. To be run on Windows, as an administrator; the VM cycle uses it to validate the extraction of a hive.

```
raw_hive_test.exe [volume] [path] [output] [--fix] [--list] [--attrs]
```

`--fix` makes the extracted hive loadable, `--list` lists a directory, `--attrs` shows the `$MFT` attributes of the file without interpreting them.

## 4.17 NTFS records: `ntfs_fixup_test`

NTFS writes a `$MFT` record or an index block sector by sector, and ends every 512-byte stride with a sequence number. WAC checks that number before restoring the real bytes: a record torn by an interrupted write is refused. The harness builds records whose expected result is known (sound, torn in each stride, malformed arrays), then confronts WAC's verdict with an independent check on 20,000 random records placed against a forbidden page. No data is needed.

```
cd build-windows/tests && wine ntfs_fixup_test.exe
```

Reference: 44,610 checks, no failure. On a real volume, the VM cycle confirms it: `raw.log` must hold no "torn record".

## 4.18 NTFS data runs: `ntfs_runs_test`

The data runs say where a file's clusters lie; read from a deleted record, they are what an intruder or a damaged disk left. The harness decodes known lists (the Linux-NTFS

documentation's examples, a run backwards, a sparse run, the attribute's end without a zero), refuses forged lists whole (a size field beyond 8 bytes, a length of zero, a truncated run, a cluster before the volume's start), then confronts WAC with an independent decoder on 128-bit integers for 50,000 random lists placed against a forbidden page.

```
cd build-windows/tests && wine ntfs_runs_test.exe
```

Reference: 100,017 checks, no failure; the former decoder, which took a field of 15 bytes, fails 48,224 of them.

## 4.19 Deleted files: `deleted_files_test`

A recovery gone wrong gives a file that looks right: the bytes of whatever lies in its clusters now. The harness builds deleted records and a volume in memory whose outcome is known: a resident file, a contiguous one, a fragmented one with a sparse run; a cluster taken before the reading, another taken *during* it; all taken; clusters read back zero (TRIM); a valid data length shorter than the size. Inconsistent lists (a run outside the volume, two runs on the same clusters, sizes that do not match, compression, encryption, runs in an extension record) are refused before any read. Every read asked for is checked: inside the volume, on free clusters. Then 20,000 damaged records, against a forbidden page. Level 2: an INDX block built with live entries and, in its slack, one valid entry among decoys (another parent directory, an 8.3 alias, a key of the wrong length, a date of 1917, a control character) — only the valid one comes out; a released block also gives its former live entries; 4,000 damaged blocks against a forbidden page.

```
cd build-windows/tests && wine deleted_files_test.exe
```

Reference: 91,839 checks, no failure — the clusters claimed since by a later deleted file among them (known cases, 2,000 draws against a cluster-by-cluster reckoning, a hole said so); without the check of the parent directory, level 2 fails three of them; without the check of `$Bitmap` after the reading, the cluster taken during the reading is published as the file's content and the check fails. In the VM, files of known content (resident, text, unsigned executable, `notepad.exe`) are written to the disk then deleted: `check-json.py` requires a content declared `Recovered` to have the SHA-256 Windows computed before the deletion, and the authentic executable to be listed without being copied; a fifth witness is written over after its deletion, by a file deleted in turn: never "Recovered" with another content. For level 2, 150 files are moved from a folder to another: at least one former name must come out of the slack of the first one's index, with the record and current path `mft.json` gives the file.

## 4.20 Carving: `carving_test`

A carved file has only its bytes: cut one byte short or long, it looks like the file and is not. The harness builds a PE (two sections, a signature), a ZIP (one file, a comment) and a PDF (an xref table) of known size, and their forged variants (unknown machine, 200 sections, a section off its alignment, a signature inside the sections, a central directory elsewhere than

the end record says, a `startxref` pointing at nothing); then 30,000 damaged heads against a forbidden page; then real files given as arguments, which must come out at their exact size.

```
cd build-windows/tests && wine carving_test.exe 'Z:\...\WAC.exe' 'Z:\...\documentation_FR.
    pdf' 'Z:\...\kape.zip'
```

Reference: 96,760 checks, no failure; WAC.exe and WAC32.exe (with their COFF symbol table), a PDF manual, an archive of 140 MB to the byte. In the VM (`WAC_DELETED_LEVEL=3`, XP and Windows 11), an executable and a PDF written, truncated to nothing then deleted must be carved with the SHA-256 of their writing; on the FAT32 and exFAT keys, a deleted text must be read back with its own.

## 4.21 `$MFT` and change journal: `mft_test`, `usn_journal_test`

An offset error in reading the `$MFT` or the change journal gives dates, names and references that are valid and false: no syntax check would see it. Both harnesses therefore compare WAC to an independent reference.

`mft_fixtures.sh` formats an NTFS volume in a file with `mkntfs`, drops files in it (resident, non-resident, a non-ASCII name) with `ntfscp`, without mounting or privilege, dumps its `$MFT` with `ntfscat`, then has each record described by `ntfsinfo` — ntfs-3g, another implementation of NTFS. `mft_test` compares, record by record, the name, the parent, the size, the eight dates and the named streams read by WAC to that description. `usn_journal_test` builds records of versions 2 and 3 with structures whose layout comes from the compiler (MinGW's `USN_RECORD`; version 3, which MinGW does not declare, from Microsoft's documentation, its offsets checked at compile time), lays them out as a journal the way NTFS leaves it (pages, zeros, a range record, a leading sparse region), checks the numbers, the reasons and the paths, then decodes every record damaged at random against a no-access page.

```
bash sources/mft_fixtures.sh /tmp/mft          # ntfs-3g: mkntfs, ntfscp, ntfsinfo
cd build-windows/tests
wine mft_test.exe "Z:/tmp/mft"
wine usn_journal_test.exe "Z:/tmp/mft"
```

Reference: 278 checks for `mft_test`, 27 for `usn_journal_test`, no failure.

## 4.22 Raw reading on a real volume: `raw_volume_test.sh`

The directory index names a file by its record number *and* a sequence number, which NTFS increments every time it frees the record. On the running system a file can be deleted between the listing of its directory and its read, and its record given to another file: WAC used to read it anyway, and sealed that other file's content under the first one's name (an `SRU*.log` holding a hive's `hbin` blocks, found in the test VM). The script serves the volume of `mft_fixtures.sh` to Wine as drive `T:`, in a Wine prefix of its own, and has `raw_hive_test` read it as WAC reads a disk: a file whose bytes must be those `ntfscat` reads, then the same file once its record has another sequence number than its index entry — refused, no copy left.

```
bash sources/mft_fixtures.sh /tmp/mft
bash sources/raw_volume_test.sh /tmp/mft build-windows/raw_hive_test.exe
```

Reference: 3 checks, no failure; the reader before the fix fails 2 of them.

## 4.23 Outlook's mail: `pst_test`

A PST read wrong — a block decrypted with the wrong table, a heap item cut one byte off, a compressed block inflated short — still gives valid JSON. `pst_fixtures.sh` downloads, pinned by their SHA-256, java-libpst's test files (Apache 2.0: two Unicode PSTs and an OST of Outlook 2013, 4 KiB pages and compressed blocks) and a PST of the EDRM Enron v2 corpus (CC-BY 3.0 US), and has each message read by pypff, libpff's reader written independently of WAC. `pst_test` compares, message by message: subject, sender, delivery and submission times, the name, size and SHA-256 of every attachment, the SHA-256 of the body.

```
PYTHON=vmtest/tools/pff-venv/bin/python bash sources/pst_fixtures.sh /tmp/pstfx
cd build-windows/tests && wine pst_test.exe Z:/tmp/pstfx
```

Reference: 1,800 checks, no failure; the 256 messages pypff reads identical, and the 4 items of the Enron PST pypff cannot read read by WAC. With the decryption removed, 253 failures.

## 4.24 Internet Explorer's indexes: `ie_index_dat_test`

Checks the decoder of Internet Explorer 5 to 9's `index.dat` files (Windows 7 and earlier) on four real files of the plaso project's public test data — history, cache, daily history, and a cache of over a thousand records with redirections, dropped entries (LEAK) and recovered ones. The reference is `pymsiecf`, libmsiecf's reader, written independently of WAC: every record it gives must come back with the same fields (URL, file, cache directory, times, expiry, last check, hits, data size), and none more. This test caught a real defect: a recovered record, partly overwritten by a newer one, gave the other's data as its own. Then every file is decoded truncated and damaged at random, against a no-access page on both sides; finally the conversion (container, account and page, cached file, headers, times) is checked.

```
pip install libmsiecf-python              # pymsiecf, for the reference
PYTHON=python3 sources/ie_index_fixtures.sh /tmp/iefx
cd build-windows/tests && wine ie_index_dat_test.exe Z:/tmp/iefx
```

The files are not versioned: the script downloads them and refuses them if their SHA-256 differs. Reference: 22 checks, no failure (1,110 records identical to `pymsiecf`'s).

## 4.25 Windows Timeline: `activities_cache_test`

Checks the reading of `ActivitiesCache.db` on a real Windows 10 database of the plaso project's public test data (112 activities: applications, pages, foreground time). The reference is Python's `sqlite3`, each column converted by the rules of `activities_cache.h`

written again in the script, independently of WAC: Unix times in UTC, GUIDs in .NET's byte order, JSON payloads parsed, null values left out. Every row must come back with the same columns and values, and none more; the local forms of the dates are not compared (they depend on the examined machine's time zone). Then a truncated copy is read without a crash. Reintroducing a one-hour shift in the conversion of the dates makes 113 checks fail.

```
sources/activities_cache_fixtures.sh /tmp/acfx
cd build-windows/tests && wine activities_cache_test.exe Z:/tmp/acfx
```

The database is not versioned: the script downloads it and refuses it if its SHA-256 differs. Reference: 118 checks, no failure.

## 4.26 Small file artefacts: `file_artefacts_test`

Checks the decoders of the PowerShell history, of `setupapi.dev.log`, of the recycle bin's `$I`, of `hosts`, of `Report.wer` and of Defender's MPLog on inputs written from each format's description, with the expected result: a command on two lines joined, a section title split into operation and subject, 31 February refused as a date, a `$I` of version 1 and 2 read, a `$I` truncated or of unknown version refused, a path length that lies bounded by the file, a problem signature ordered by its index whatever the file's order, a date without `Z` not taken as UTC. For `$Secure:$SDS`: a stream built block by block, where the mirror block is skipped even when its entry looks consistent, a repeated `SecurityId` kept once, a length leaving the block refused, a damaged hash flagged (`HashMismatch`), the owner and the ACE decoded, every read against a no-access page on both sides. For `$Boot`: geometry and serial read, a sectors-per-cluster count above 0x80 taken as a power of two, a sector without 0x55AA, of another file system, of 100 bytes or truncated refused; for `$Max`: the four fields, a stream too short refused. For a DPAPI master key: header read, block sizes beyond the file or wrapping when added refused, a truncated header refused; `Preferred` given in the form of the key files' names. For `UserShellFolders`: `%USERPROFILE%` replaced by the profile whatever its case, a path on another disk kept, a share or another user variable not followed. For the SQLite reading: a database in WAL mode whose writer is still open, as a running browser leaves it, read on its private copy with its log's rows (UTF-8 text, integer, NULL, BLOB), the database and its log unchanged, the copy deleted, a query of another schema and a file that is no database refused. For the Chromium history: a `History` built with Chromium's tables, a typed visit then a link citing it, a redirected download whose chain comes back in its order, a searched term; transition types and qualifiers, Chromium times converted. For Firefox: `places.sqlite` and `formhistory.sqlite` built with Firefox's tables, a download visit citing the visit it came from, a download's destination decoded from its `file:///` URI (UTF-8, a space), its end in milliseconds, an address bar input and a search bar entry. The VM has no Firefox: this test is its only check. Saved logins and cookies of Chromium and Firefox: site, account, dates (Firefox cookies' expiry in seconds), and no password nor cookie value in the JSON. No data needed.

```
cd build-windows/tests && wine file_artefacts_test.exe
```

Reference: 174 checks, no failure (the second-lot registry values included: TrustRecords, firewall rule, certificate Blob; and the thumbnail caches: a real entry header of the VM and

the checksum Windows wrote for it, a cache built here, forged lengths; Thunderbird's mail: an mbox written here, encoded words, quoted-printable, base64, an RFC 2231 name, a nested multipart or one without boundary, a date with its offset). In the VM cycle, `place-file-artefacts.ps1` places witnesses whose content is known (a history, a file sent to the bin, a hosts entry, a startup script) and `check-json.py` finds them in the JSON.

## 4.27 Signatures: `authenticode_test`

Verifies the catalogs of a folder, then gives a verdict per file; its verdicts are compared with those of `Get-AuthenticodeSignature` on the same files, in the VM. The only gap that matters: a file WAC accepts and Windows refuses.

```
authenticode_test --catalogs <folder>
authenticode_test --catalogs <folder> --files <list>
```

## 4.28 Configuration: `yaml_light_test`, `config_test`

`yaml_light_test` (native, libyaml included, under AddressSanitizer and UBSan): exact readings (nested sections, quotes, comments, byte order mark, empty file), refusals with the right line (duplicate key, list outside a key, list in a list, mapping in a list, anchor, alias, tag, second document, tab, quote left open, nesting too deep), lists of values read exactly (the deleted files' excluded folders), then 3,000 texts damaged at random; seven mutations all detected. `config_test` (under wine): each setting applied, each kind of mistake refused with its line, and the reference file of `--write-config` read back as it describes itself; three mutations detected. In the VM, a collection driven by a `wac.yml`, without any option, is checked against the file recorded in `investigation.json` (`check-json.py --configuration`).

## 4.29 Third-party chains: `third_party_chain_test`

Each rule of the check of third-party chains, on chains made by openssl to break it (`third_party_chain_fixtures.sh`): a sound chain accepted; the root absent, excluded from code signing, distrusted; an authority's key disallowed by Microsoft; an authority revoked according to the CCADB; an issuer without `cA`; a signer for the web only; an authority with the right name and another key; an ECDSA authority (not verified). Then the time a chain is judged at, with a real RFC 3161 token made by `openssl ts` and the collection's time moved to 2040: a signer valid 2020 to 2035 accepted without time stamp today, refused in 2040, accepted in 2040 when stamped today; a token over another signature, or altered, refused; a time stamping authority under a root not trusted for it, refused; a root distrusted after 2030 accepted for a signature stamped before, refused otherwise. Last, revocation, with real CRLs made by `openssl ca`: a sound chain accepted with the authority's and the root's CRLs; without them, with the authority's missing or forged (right issuer name, another key), refused as not verifiable; a CRL found through the CCADB when the certificate's address is missing, the certificate's own prevailing; a signer revoked for a compromise refused

even when stamped before; one revoked as superseded after the time stamp accepted when stamped, refused otherwise. Each rule disabled in turn fails the test (twenty-two mutations). The time stamps of the real installers agree to the second with an independent reading (`openssl ts` for RFC 3161 tokens, DER for counter-signatures). Real signatures are cross-checked by `authenticode_test --trust`, against `openssl verify` with the set's `roots.pem` (installers of Git, Notepad++, PuTTY, Python, VLC; the whole set and the set without the Sectigo roots); in the VM, no chain WAC trusts is untrusted by `Get-AuthenticodeSignature`.

```
./third_party_chain_fixtures.sh /tmp/chain
/tmp/third_party_chain_test /tmp/chain
```

## 4.30 Vulnerable drivers: `version_info_test`, `vulnerable_drivers_test`

`version_info_test` checks the reading of the version resource (original name, internal name, product, description, version) against Windows' (PowerShell's `VersionInfo` in the VM) on 58 binaries: versions and texts identical, except the texts a localised binary takes from its `.mui` satellite — which the blocklist does not read —, then truncated and damaged resources (11,852 checks). `vulnerable_drivers_test` takes each rule alone: an Authenticode or file fingerprint, a denied authority in the chain, narrowed to the signer, the WHQL manufacturer, the original name, the product, the version bounds; a binary without version resource (the file rule does not apply, as with Windows) or not read (it applies) — nine mutations all detected. On real LOLDrivers samples, WAC's Authenticode digest is their "Authentihash" (4 out of 4), and two vulnerable drivers whose chain holds are refused; in the VM, a vulnerable driver placed there (never loaded) must be recognised.

```
g++ -std=c++17 -g -fsanitize=address,undefined -I. pe_resource.cpp \
    version_info.cpp version_info_test.cpp -o /tmp/version_info_test
/tmp/version_info_test <reference> <folder>
```

## 4.31 Trust lists: `inflate_test`, `cab_test`, `zip_test`, `csv_reader_test`, `trust_list_test`

`--update-trust` decompresses and reads what Microsoft publishes; an error would give a valid and wrong output. Each link therefore has its independent reference, and is built on Linux with AddressSanitizer and UBSan:

- `inflate_test`: DEFLATE, checked against zlib on 49 streams, then streams truncated and damaged at random (fixed seed);
- `cab_test`: MSZIP cabinets, extraction identical byte for byte to `cabextract`'s on the real `authrootstl.cab` and `disallowedcertstl.cab` (1,134 checks);
- `zip_test`: ZIP archives, extraction identical to Python's `zipfile` on Microsoft's real vulnerable driver list and two synthetic archives (stored, compressed, comment,

folders, empty file); a damaged archive never returns an altered content, refused by its CRC-32 (1,717 checks);

- `csv_reader_test`: the CCADB CSV report, read field by field as Python's `csv` module reads it (10,284 records, SHA-256 of every field), then doubled quotes, line breaks in a field, CRLF, a quote left open, a short record;
- `trust_list_test`: the counts of `authroot.stl` (roots for code signing, for time stamping, and for code signing not distrusted), computed apart by a DER reading in Python; the certificates Chromium blocks (`net/data/ssl/blocklist`), recognised by `disallowedcert.stl`; a list with one byte changed, refused; both lists read by eight threads at once, the chain cache empty, as the analysis threads of `--collect --binary` do. Built again with `-fsanitize=thread` (run through `setarch -R`, which ThreadSanitizer needs on recent kernels), it reported eleven data races on that cache before its lock.

```
g++ -std=c++17 -g -fsanitize=address,undefined -I. authenticode.cpp rsa.cpp \
    sha.cpp quickdigest5.cpp trust_list_test.cpp -o /tmp/trust_list_test
/tmp/trust_list_test authroot.stl 313 335 145 disallowedcert.stl blocklist 36
```

Reference (lists of 25 August 2026 and 5 September 2025): 80 checks, no failure. This test established what `disallowedcert.stl` holds, which Microsoft does not document: its 16-byte entries are the MD5 of the **public key** (not of the signed part, as the property's name suggests), its 48-byte entries the SHA-384 of the signed part.

The end-to-end result is checked in the VM: the 562 roots `--update-trust` writes are identical, byte for byte, to those `certutil -generateSSTFromWU` produces. The vulnerable driver lists are cross-checked by a Python reading of the downloaded XML and JSON: the same 1,086 Microsoft fingerprints, 217 denied signers, 8,151 LOLDrivers fingerprints. The CRLs are checked with openssl: each one reads, and its issuer is the authority the CCADB associates with it (387 out of 387).

CHAPTER 5

# The robustness harnesses

## 5.1 The principle

A parser reads at offsets the file itself declares. On a truncated or forged file, a read past the buffer does not crash on its own: it reads the neighbouring memory and publishes it, in valid JSON. The harnesses make it crash: each input is copied against a forbidden memory page (`PAGE_NOACCESS`), once *ending* against it (reads past the end), once *starting* right after it (reads before the start, for instance at a negative offset taken from the data). A single byte read outside the input raises an access violation.

For each file given, the harness parses:

- the whole file;
- its truncations (every length for small files);
- copies with bytes overwritten at random, with a fixed seed: a failure reproduces identically.

## 5.2 Shortcuts: `lnk_test`

```
cd build-windows/tests
wine lnk_test.exe a.lnk b.lnk ...
```

Where to find files: the `Recent` folder and the Start menu of the VM, or those of a collection (`Users\<name>\AppData\Roaming\Microsoft\Windows\Recent`). Accented or non-Latin names travel badly through the `wine` command line: copy the files under numbered names.

## 5.3 Jump lists and Prefetch: `parsers_test`

```
parsers_test.exe jumplist-auto   <file.automaticDestinations-ms> ...
parsers_test.exe jumplist-custom <file.customDestinations-ms> ...
parsers_test.exe prefetch        <file.pf> ...
parsers_test.exe hive            <hive> ...
```

The `hive` kind tries the hive reader: each input is opened from memory, against the forbidden page, and the whole tree is walked (subkeys, values, `ORQueryInfoKey`). A subkey pointing back at one of its ancestors is refused by the reader: without that, a recursive walk would never end.

Jump lists are tested under `wine`, like `lnk_test`. Prefetch files are tested **in the VM**: they are compressed (XPRESS Huffman), and only the `ntdll` library of Windows decompresses them. The harness decompresses each file before trying it: it is WAC's parser it aims at, not Windows' decompressor.

For the 307 Prefetch files of a real machine (over a million inputs), count about two hours in the VM: run the test as a detached task, in batches, and follow its log. Example PowerShell script, put into the VM with `qga.py write`:

```
$files = Get-ChildItem C:\wactest\pt\pf\*.pf | ForEach-Object { $_.FullName }
$log = 'C:\wactest\pt\pf_results.txt'
for ($i = 0; $i -lt $files.Count; $i += 20) {
  $batch = $files[$i..([Math]::Min($i + 19, $files.Count - 1))]
  Add-Content $log (& C:\wactest\pt\parsers_test.exe prefetch @batch 2>&1)
  Add-Content $log ("batch " + $i + " exit=" + $LASTEXITCODE)
}
Add-Content $log 'DONE'
```

## 5.4 Reference

Results obtained on a real machine, each input parsed on both sides of the forbidden page:

| Files | Inputs | Where |
|---|---|---|
| 220 shortcuts | 546,214 | `wine` |
| 48 automatic jump lists | 175,585 | `wine` |
| 23 custom jump lists | 79,036 | `wine` |
| 307 Prefetch files | 1,246,491 | VM, about 2 h |
| 4 registry hives | 16,207 | `wine` |

CHAPTER 6

# Cross-checking on a real collection

A freshly installed VM produces no fragmentation, no old jump lists, no exotic file names. Several defects only showed on a real collection, confronted with an independent source:

- **jump lists / `olefile`**: the streams and the DestList, read again by an independent OLE reader, must give exactly the same entries (numbers, dates, paths). That is what showed the streams are named “1” to “f”, not “01” to “0f”;
- **shortcut / DestList**: in a jump list, the target of a shortcut and the path of its DestList entry are two records of the same file. Their disagreement revealed a wrong code page (“d\x92orientation”) and an ignored Unicode path;
- **events / `python-evtx`**: record-by-record comparison (`evtx_test --dump`);
- **signatures / `Get-AuthenticodeSignature`** (`authenticode_test`);
- **old collection / new one**: decode the same files with the current code and compare, after normalisation, with the output of a previous version.

> **Warning**
> A real collection is processed locally. If a hive must be confronted with Microsoft’s reader (`offline_registry_test`), it may be copied into the test VM, then deleted as soon as the test is over. No copy stays in the temporary working folder, and none of it enters the repository.

CHAPTER 7

# Reading a failure

## 7.1 Exit codes

| Code | Meaning |
|---|---|
| 0 | everything passed |
| 1 | a check failed, or an input file is unreadable |
| 2 | wrong usage (arguments) |
| 3 | `parsers_test`: a parse did not end within 60 s (watchdog) |
| `0xC0000005` (-1073741819) | access violation: a read outside the buffer |

## 7.2 Locating a read outside the buffer

Under `wine`, a fault prints the call stack, with addresses relative to the program (`lnk_test+0x485bb`). On Windows, `parsers_test` itself prints the input at fault and the faulting address:

```
FAILED  exception 0xC0000005 at exe+0x78b70, address read 0x159deb4200c:
        C:\wactest\pt\pf\CMD.EXE-0BD30981.pf truncated at 192 (guard after)
```

The address is translated into a function with the program's symbol table (base address `0x140000000`):

```
x86_64-w64-mingw32-nm -C parsers_test.exe | grep -i " t " | sort > syms.txt
# look for the symbol whose address is just below 0x140000000 + 0x78b70
x86_64-w64-mingw32-objdump -d --start-address=0x140078b40 \
    --stop-address=0x140078b80 parsers_test.exe
```

The disassembled instruction gives the offset read (here `0xcc(%r14)`); it is then matched, in the source code, with the field read at that position. Then replay the exact input (the truncation or corruption shown) to check the fix, and the whole series after it.

## 7.3 An endless loop

If `parsers_test` stops on "no end after 60 s", the input named keeps a parser running for ever — typically a cyclic sector chain in an OLE container. It is a defect just like a read outside

the buffer: a forged file would be enough to block a collection.